PROJECT REPORT: CONTACT MANAGER

Team No: 2

TEAM LEADER

- **Name: Padhmapriya D**

  NM ID: 4E08C53EBFAB2BB87434A28803F55C7E

TEAM MEMBERS

- **Name: Jyothisha S R**

  NM ID: 71D39EB9B369C2B9F1D45C8B3D316078

- **Name: Karthika K**

  NM ID: 9FD8E6E9603E782A93145081C81D0CE4

- **Name: Kaviya Shree S**

  NM ID: C1CDDCD0A080C20AFAA481167FC1D118

Technical Documentation for a Secure Contact Management System

---

1. Abstract

The **ConnectHub Contact Management System** is a high-performance backend solution engineered to mitigate the challenges of fragmented contact information. Developed using the Node.js ecosystem, the system provides a secure, centralized API for managing extensive personal and professional networks. By leveraging a three-tier architecture and NoSQL database technology, ConnectHub ensures high availability, strict data isolation, and seamless cross-platform integration.

2. Introduction

In the contemporary digital landscape, the proliferation of communication channels has led to "data silos," where contact information is scattered across various devices and disparate applications. ConnectHub is an enterprise-grade backend service designed to consolidate this information into a single, accessible, and highly secure platform. Built on the Express.js framework, it offers a RESTful interface optimized for modern web and mobile application consumers.

3. Problem Statement

End-users frequently encounter difficulties managing diverse contact lists across multiple platforms, resulting in data redundancy, information loss, and significant security risks. Existing solutions often lack advanced privacy-centric features or fail to provide a scalable mechanism to categorize and search through high volumes of complex professional data.

4. Objectives of the System

- **Security & Data Privacy:** Implementation of rigorous authentication protocols to ensure complete data isolation.
- **Data Lifecycle Management:** Facilitating comprehensive CRUD (Create, Read, Update, Delete) operations with high reliability.
- **Organizational Utility:** Providing a flexible, array-based tagging system for logical contact categorization.
- **Performance Optimization:** Delivering sub-second search and filtering capabilities for optimized data retrieval.

5. Scope of the Project

The scope of this project is strictly defined within the **Backend-as-a-Service (BaaS)** layer. It encompasses user account lifecycle management, secure session handling via JSON Web Tokens (JWT), database schema architecture for contact storage, and the complex business logic required to process dynamic search queries and real-time data updates.

6. Target Users

- **Independent Freelancers:** For managing intricate client relationship data.
- **SME Owners:** For tracking vendor profiles and customer lead information.
- **Young Professionals:** For building and maintaining career-focused networking directories.
- **General Consumers:** For a secure, private, and searchable personal directory.

7. Technology Stack

- **Runtime Environment:** Node.js
- **Backend Framework:** Express.js
- **Database:** MongoDB (NoSQL)

- **Object Data Modeling (ODM):** Mongoose
- **Security:** JWT (Stateless Authentication) & bcrypt (One-way Password Hashing)
- **Development Suite:** VS Code, npm, Git, Postman/Thunder Client

8. System Architecture

ConnectHub utilizes the **Model-View-Controller (MVC)** architectural pattern to ensure modularity:

- **Models:** Define the BSON data structure and interface with the MongoDB layer.
- **Controllers:** Process incoming HTTP requests and execute core business logic.
- **Routes:** Map specific HTTP methods and endpoints to their respective controller functions.
- **Middleware:** Handle cross-cutting concerns such as authentication guards and centralized error handling.

9. Database Design

The system employs a non-relational, document-oriented storage model via MongoDB.

9.1 Users Collection

Field	Type	Constraint
_id	ObjectID	Primary Key (Auto-generated)
name	String	Required
email	String	Required, Unique, Indexed
password	String	Hashed via bcrypt
timestamps	Date	CreatedAt / UpdatedAt

9.2 Contacts Collection

Field	Type	Constraint
_id	ObjectID	Primary Key
userId	ObjectID	Foreign Key (Ref: User)
name	String	Required
phone	String	Validated String
tags	Array	Indexed for Search

10. ER Diagram Explanation

The system architecture follows a **One-to-Many (1:N)** relationship model. A single **User** entity can maintain an unlimited number of **Contact** documents. However, each individual Contact record is strictly bound to its creator's unique userId. This architecture ensures a "walled garden" approach where one user is programmatically prevented from querying or modifying the contacts of another.

11. System Modules

1. **Authentication Module:** Manages user onboarding, credential verification, and token issuance.
2. **Contact Management Module:** Governs the core data lifecycle and schema validation.
3. **Search & Indexing Module:** Executes optimized regex-based queries for real-time results.

12. Authentication System

ConnectHub implements a stateless authentication mechanism:

- **Secure Registration:** Passwords undergo a salt-and-hash process using **bcrypt** before database persistence.
- **Verification:** Upon login, the system validates the hashed credentials and issues a signed **JWT**.
- **Authorization:** Custom middleware intercepts requests to protected routes, verifying the token's validity before granting access to user-specific resources.

13. Contact Management System

The CRUD module ensures all data complies with strict Mongoose validation rules:

- **Data Integrity:** Validates email syntax and phone number strings before insertion.
- **User Isolation:** Every operation (Read/Update/Delete) is automatically filtered by the userId extracted from the authenticated JWT payload.

14. Search and Tagging System

The system leverages MongoDB's powerful querying capabilities to provide:

- **Multi-Criteria Search:** Users can perform partial-string searches across name and email fields simultaneously.
- **Categorical Tagging:** Dynamic filtering allows users to isolate contacts based on array-resident tags (e.g., "Work," "Urgent," or "Personal").

15. API Endpoints

Method	Endpoint	Authentication
POST	/api/auth/register	Public
POST	/api/auth/login	Public
GET	/api/contacts	Private (JWT Required)
POST	/api/contacts	Private (JWT Required)
PUT	/api/contacts/:id	Private (Owner Only)
DELETE	/api/contacts/:id	Private (Owner Only)

16. Folder Structure

```
📁 ConnectHub-Backend
└── 📁 server/
    ├── 📁 config/          # db.js, passport.js (Configuration)
    ├── 📁 controllers/     # authController.js, contactController.js
    ├── 📁 middleware/      # authMiddleware.js, validation.js
    ├── 📁 models/          # User.js, Contact.js
    ├── 📁 routes/          # authRoutes.js, contactRoutes.js
    ├── 📄 .env             # Environment Secrets (Private)
    ├── 📄 package.json     # Project Dependencies & Scripts
    └── 📄 server.js        # Main Entry Point & App Setup
```

17. Implementation Steps

1. **Environment Setup:** Node.js initialization and .env configuration for sensitive keys.
2. **Schema Modeling:** Constructing Mongoose models for User and Contact entities.
3. **Security Integration:** Developing the bcrypt hashing and JWT middleware layers.
4. **Logic Development:** Creating controllers for authenticated CRUD and search logic.
5. **Integration Testing:** Mapping controllers to routes and verifying via Postman.

18. Testing and API Validation

Comprehensive testing was performed using **Postman** and **Thunder Client**. All endpoints were validated against standard HTTP status codes:

- 201 Created: Successful registration/contact creation.
- 200 OK: Successful login and data retrieval.
- 401 Unauthorized: Denied access to protected routes without a valid token.
- 404 Not Found: Identification of non-existent resources.

19. Advantages of the System

- **Horizontal Scalability:** The NoSQL structure allows for rapid data growth.

- **Advanced Security:** Enterprise-grade hashing and tokenization protect user assets.
- **Clean Architecture:** The MVC pattern makes the system easy to maintain and extend.

20. Limitations

- **Headless Service:** Currently lacks a graphical user interface (Frontend).
- **Storage Constraints:** No current support for binary file (image) uploads.

21. Future Enhancements

- **Client Integration:** Development of a **React.js** frontend dashboard.
- **Binary Media Handling:** Integration with **Cloudinary** or **AWS S3** for contact photos.
- **Automated Sync:** Integration with third-party contact APIs (Google/Outlook).

22. Conclusion

The **ConnectHub Contact Management System** successfully delivers a professional, secure, and highly efficient backend framework. By prioritizing architectural integrity and security, the system provides a scalable foundation ready for production-level contact management applications.

Prepared by: Padhmapriya D *Lead Developer*